

Multi-Domain Contract

Schema-driven domain onboarding for the AgenticAI Manufacturing Hybrid GraphRAG pipeline. What's domain-specific, what's auto-derived, what the schema YAML can now control without any Python edit, and what's still global.

1. The promise

Adding a new domain is a **two-path** change:

- **Author** a single YAML at `schemas/<domain>.yaml`
- **Drop** source documents into `doc_pipeline/input_docs/<domain>/`

Everything else — UI selector, FastAPI routing, Qdrant collection, KG snapshot, FAISS index, per-domain LLM persona, HITL escalation vocabulary, clarifier intent / slot prompts, structured procedure drafter opt-in — is derived from the schema YAML at startup. Restart the API and the new domain appears in every surface.

Manufacturing remains byte-identical: every schema-driven path falls back to the legacy module-level constants when the YAML block is absent, so no regression risk for the existing domain.

2. Files you author per domain

Path	What goes in it	Required?
<code>schemas/<domain>.yaml</code>	The whole domain contract — ontology, prompts, safety keywords, clarifier vocabulary, HITL escalation, etc.	Yes (single file)
<code>doc_pipeline/input_docs/<domain>/</code>	Raw PDFs / TXT / XLSX files for ingestion. Subfolders named <code>management/</code> / <code>internal/</code> / <code>external/</code> are also allowed.	Yes (dependencies)

3. Auto-derived artifacts (don't touch)

Path / resource	Generated by
<code>data/processed/knowledge_graph.<domain>.json</code>	KG builder on first ingest (<code>config.kg_path</code>)
Qdrant collection <code><domain>_corpus</code>	Vector indexer (<code>config.qdrant_collection</code>)
FAISS index name <code><domain>_index</code>	Embedding pipeline (<code>config.index_name</code>)
<code>/api/domains</code> entry	<code>config._discover_domains()</code> scans <code>schemas/*.yaml</code> at startup
Streamlit + Next.js header switcher	Both UIs read from <code>/api/domains</code>
Per-domain ManufacturingPipeline instance	<code>api/server.py</code> builds one pipeline per discovered domain

4. Anatomy of schemas/.yaml

Each block below is independent. Fill in the ones you need; omitted blocks fall back to manufacturing-flavoured defaults.

4.1 Identity & UI

Block	Effect when set	Fallback when omitted
domain: <id>, version: 1	Required identity	—
display: {label, emoji, color}	Shows up in selectors / header chips	Title-cased id +  + slate-500
placeholder:, empty_state:, example:	Chat input copy + landing page	Generic fallback

4.2 Knowledge graph ontology

Block	Effect when set	Fallback when omitted
entity_types: [...]	Closed/regex/open-vocab entity types. Drives KeywordExtractor	KG core extractor and schema validation fully
edge_types: [...]	Edges the KG will accept (with source/target type constraints and cardinality)	Same as entity
traversal_routes: {...}	Named walks the orchestrator uses for query-driven subgraph exploration	Routes are silently skipped
gap_thresholds: {...}	KG gap detector's low-confidence / retrieval floors.	Defaults from config

4.3 LLM personas & prompts (NEW)

Read by `core/domain_prompts.get_prompt()`. Every block below is optional; any unset key falls back to the module-level constant in the matching `core/*_py` file (manufacturing-flavoured).

Block (under prompts:)	Consumed by	Effect
persona	free-form, used by your own templates	Short label — "aviation maintenance copilot"
answer_system	<code>core/orchestrator.py</code> , <code>pipeline/langgraph_orchestrator.py</code>	Free-system prompt for the free-form answer LLM call
retry_system	both orchestrators (critic-rejected retry path)	Improved-answer system prompt (uses {critic_feedback})
critic_rules	<code>core/critic.py</code>	The quality critic's evaluation criteria
procedure_system	<code>core/procedure_drafter.py</code>	Structured drafter persona; controls safety-precondition phrasing (L)
cause_rank_system	<code>core/cause_ranker.py</code>	Root-cause ranking persona (uses {top_k}, {taxonomy_clause})
classify_system	<code>core/query_formatter.py</code>	Intent classifier LLM prompt
risk_grader_system / risk_grader_user	<code>core/criticality_classifier.py</code>	Tier-2 HITL risk grader prompts

4.4 Safety / HITL escalation (NEW)

Block	Consumed by	Effect
safety.high_risk_keywords: [...]	<code>core/criticality_classifier.classify()</code>	Domain-specific keyword list that escalates a query/answer to a human

4.5 Clarifier vocabulary (NEW)

Read by doc_pipeline/clarifier_agent.py via the clarifier: top-level block.

Sub-block	Effect when set	Layering behaviour
equipment_patterns: [{pattern, type}]	Extra regexes for entity extraction (e.g. WO:ASRS:NNM, ENG:360:AA1).	Appended
part_number_patterns: [{pattern, type}]	Domain-specific part-number regexes.	Appended
supplier_names: {key: label}	Dictionary lookups for supplier entities.	Merged (schema wins on collision)
metric_names: {key: label}	EGT/CHT/MP/RPM for aviation; OEE/MTBF for manufacturing.	Merged
department_names: {key: label}	Domain-specific department vocabulary.	Merged
intent_patterns: [{intent, patterns, boost}]	Extra intent regexes — e.g. aviation's "mag drop" boosts "mag drop" over "mag drop" confidence.	Appended
slot_templates: {INTENT: [...]}	Per-intent clarification prompts the user sees ("Which engine?")	Merged

4.6 Query auto-correction

Read by doc_pipeline/query_correction.py via the corrections: top-level block.

Sub-block	Effect when set
corrections.acronyms: {key: expansion}	Domain acronym expansions (asrs, far, amt, mel, …). Merged with manufacturing defaults; so
corrections.misspellings: {wrong: right}	Per-domain typo fixes (carberator → carburetor).
corrections.synonyms: {key: […]}	Term enrichment ("carb ice" → carburetor icing, induction icing).
corrections.vocabulary: [...]	Extra protected vocabulary so close-match correction won't rewrite domain terms.

4.7 Structured procedure drafter opt-in (NEW)

Block	Effect	Default
procedure.enabled: bool	Master switch. Set false to skip the structured drafter entirely (legal lookup, market research, medical reference)	True
procedure.trigger_intents: [...]	Substring list — drafter fires when any trigger is a substring of the false intent	False

5. Known gaps still global (not yet schema-driven)

These remain manufacturing-leaning and may need a small code change for non-adjacent domains. None of them block read-only Q&A on a new domain — they're mostly noise rather than failure paths.

Where	What's still global	When it bites
core/query_formatter.py	MANUFACTURING_ABBREVIATIONS, INTENT_PATTERNS	Only the regular fallbacks when the LLM classifier is not confident
core/cause_ranker.py	_TROUBLESHOOTING_TRIGGERS substring list gates the cause ranker	Doesn't work with only the procedure blocks but gate by a closed list
config.CAUSE_TAXONOMY	Closed list of allowed cause names (manufacturing-flavoured: Newidgmaas, leak, leak, & oil)	Not a problem, likely to be replaced or override
doc_pipeline/clarifier_agent.Intent	Fixed enum (LOOKUP / COMPARISON / TROUBLESHOOTING / ANALYSIS)	Not a problem, for domains that add features not in manufacturing
core/purchase_request.py	Assumes USD purchase-order parsing for HITL	Irrelevant but inert on non-PO domains

6. Minimum viable new domain (30-second version)

1. Create schemas/<domain>.yaml with at minimum: domain, version, display, entity_types, edge_types.
2. Drop documents into doc_pipeline/input_docs/<domain>/.
3. Restart the API. Discovery auto-registers the domain, the UI gets a new tab, chat/copilot route queries to a fresh Qdrant collection + KG, and the system answers with the manufacturing persona (fallback).
4. (Recommended) Add prompts:, safety:, clarifier:, and procedure: blocks to stop sounding like a manufacturing copilot. Author each block once; the system reads them on the next restart.

7. Worked example — aviation schema overrides

Lifted from schemas/aviation.yaml — shows the shape of every block.

```
prompts:
  persona: aviation piston-engine maintenance copilot
  answer_system: |
    You are an aviation maintenance copilot specialising in
    piston-engine diagnostics, FAA AMT chapters, and ASRS work-order
    analysis. ... [full text in schemas/aviation.yaml]
  critic_rules: |
    You are a strict quality critic for aviation piston-engine
    diagnostic answers. ... [full text]
  procedure_system: |
    Sequence the steps so safety preconditions (mag-ground, prop
    clear, fuel shutoff, master off) come BEFORE any inspection.

safety:
  high_risk_keywords:
    - emergency
    - mayday
    - in-flight
    - airworthiness
    - prop strike
    - fuel leak
    - in-flight fire
    - ad compliance

clarifier:
  equipment_patterns:
    - { pattern: '\\b(WO:ASRS:\\d+)\\b', type: work_order_id }
    - { pattern: '\\b(ENG:[A-Z0-9_-]+)\\b', type: engine_id }
  metric_names:
```

```

    egt: Exhaust Gas Temperature
    cht: Cylinder Head Temperature
intent_patterns:
  - intent: TROUBLESHOOTING
    patterns:
      - '\\b(mag.?drop|magneto drop)\\b'
      - '\\b(carb ice|carburet?or ic(?:ing|e))\\b'
    boost: 0.93
slot_templates:
  TROUBLESHOOTING:
    - name: aircraft_or_engine
      entity_types: [engine_id, equipment_id]
      required: true
      prompt: "Which engine or aircraft? (e.g., ENG:O-360-A4M, N12345)"

corrections:
  acronyms: { asrs: Aviation Safety Reporting System, amt: aviation maintenance technician }
  misspellings: { carberator: carburetor, magneto: magneto }

procedure:
  enabled: true
  trigger_intents: [troubleshoot, root_cause, diagnos, repair]

```

8. Source files implementing the contract

File	Role
schemas/<domain>.yaml	The authoring surface — everything domain-specific lives here
core/domain_prompts.py	Cached loader: get_prompt(), get_high_risk_keywords(), get_procedure_config(), procedure_s
config.py (_discover_domains)	Startup scan of schemas/*.yaml → DOMAINS, SCHEMA_PATHS, DOMAIN_QDRANT_COLLE
core/orchestrator.py, pipeline/langgraph_orchestrator.py	Read self._domain from KG; pass it to every LLM call; gate procedure drafter on procedure_sh
core/critic.py, core/procedure_drafter.py, core/cause_extractor.py, core/tri_state_classifier.py	Each takes core/domain_yaml as input, core/tri_state_classifier via get_prompt() (or keyword list v
doc_pipeline/clarifier_agent.py	IntentClassifier, EntityExtractor, SlotFiller all take domain and merge schema's clarifier: block
doc_pipeline/query_correction.py	QueryCorrector(domain=) merges schema's corrections: block